INSTITUTO FEDERAL
Rondônia

Instituto Federal de Educação, Ciência e Tecnologia de Rondônia
Campus Guajará-Mirim
Curso Técnico em Informática Integrado ao Ensino Médio

Projeto de Desenvolvimento de Sistemas

Teste de Software

Prof. Júlio Henrique da Silva Lopes

3º ano | Técnico em Informática

GUAJARÁ-MIRIM, 2026

ROTEIRO



01

Conceitos

Erro, defeito, falha, verificação, validação e depuração



02

Níveis e técnicas

Unitário a aceitação, caixa-preta e caixa-branca



03

Na prática

Casos de teste, registro de defeitos, regressão e PHPUnit

01

Conceitos de Teste

O que é testar e o que o teste consegue mostrar

O QUE É TESTAR

- **Teste de software:** executar um programa com dados escolhidos para encontrar defeitos antes do uso real.
- Compara o resultado obtido com o resultado esperado, definido pelos requisitos.
- **Limite:** o teste mostra a presença de defeitos, nunca a ausência deles (Dijkstra).
- Testar todas as entradas possíveis é inviável. Por isso escolhemos entradas com técnicas.

Um campo de senha com 20 caracteres e 95 símbolos possíveis tem mais de $\mathbf{10^{39}}$ combinações. Nenhuma equipe testa todas.

ERRO, DEFEITO E FALHA



Erro

Engano de uma pessoa

Escrever > 8 em vez de >= 8



Defeito

Problema no código

A condição errada fica no arquivo



Falha

Comportamento errado na execução

Senha de 8 caracteres é recusada

Nem todo defeito gera falha: o defeito só aparece quando a linha errada é executada com certos dados.

VERIFICAÇÃO E VALIDAÇÃO

Verificação

Construímos o produto corretamente?

- **Compara o sistema com a documentação**
- O código segue o diagrama de classes?
- O cadastro grava todos os campos dos requisitos?

Validação

Construímos o produto certo?

- **Compara com a necessidade real do usuário**
- O sistema resolve o problema definido?
- O usuário consegue usar?
- O usuário conclui a tarefa sem ajuda?

TESTE E DEPURAÇÃO



Teste

- Objetivo: revelar falhas
- Feito com dados planejados
- Pode ser feito por quem não escreveu o código
- Resultado: lista de falhas encontradas



Depuração

- Objetivo: localizar e corrigir o defeito
- Feita a partir de uma falha conhecida
- Feita por quem conhece o código
- Resultado: código corrigido

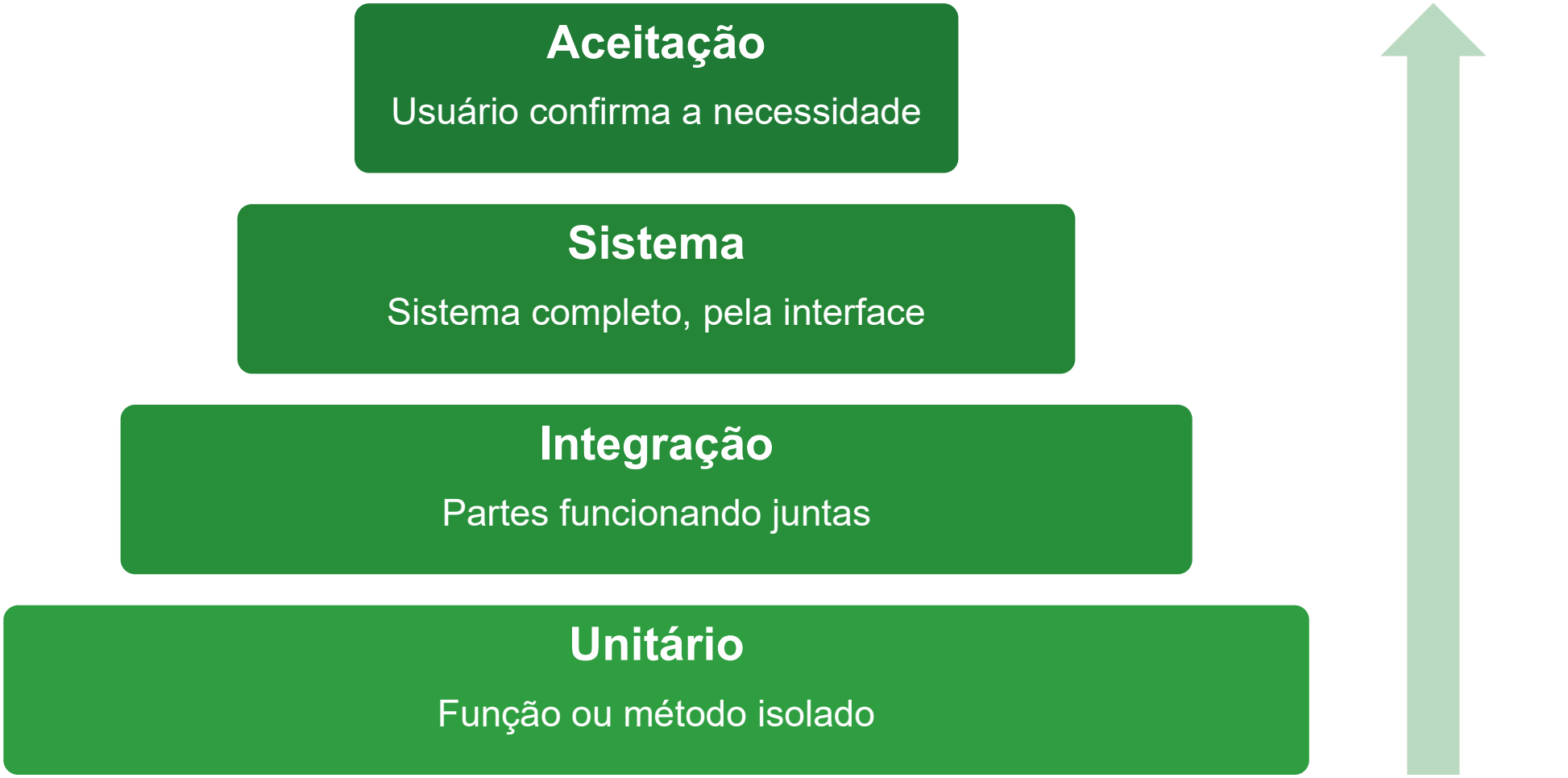
Ferramentas de depuração em PHP: `var_dump()`, `error_log()` e o Xdebug, com pontos de parada.

02

Níveis e Técnicas de Teste

Em que momento testar e como escolher os dados

NÍVEIS DE TESTE



A base é larga: muitos testes unitários, rápidos e baratos. O topo tem poucos testes, lentos e caros.

TESTE UNITÁRIO

- **Unidade:** a menor parte testável, normalmente uma função ou um método.
- Executado isolado do banco e da interface, pelo próprio desenvolvedor.
- Automatizado: roda em segundos e pode ser repetido a cada alteração.

```
function validarComprimentoSenha(string $s): bool {
  $t = mb_strlen($s);
  return $t >= 8 && $t <= 20;
}
```

Testes desta unidade: **`validarSenha('1234567')`** deve retornar false e **`validarSenha('12345678')`**, true.

TESTE DE INTEGRAÇÃO

- Cada parte pode funcionar sozinha e falhar quando é ligada à outra
- **Verifica se componentes testados separadamente funcionam juntos**
- **Foco:** as interfaces entre as partes, como controller e model ou model e banco.
- Usa um banco de testes separado, que pode ser apagado e recriado.

Defeitos típicos encontrados

Nome de coluna diferente entre SQL e banco	Tipo incompatível, como texto em campo numérico	Método chamado com parâmetros na ordem errada

TESTE DE SISTEMA

- Testa o sistema complete como o usuário usa, pela interface, em ambiente parecido com o de uso real.
- Baseado nos requisitos. Normalmente usa **técnicas de caixa-preta**.

Funcional

Cadastro, login e listagem produzem o resultado esperado?

Desempenho

A listagem com 1.000 registros abre em tempo aceitável?

Segurança

Uma página interna abre sem login?

Compatibilidade

O layout funciona no Chrome, no Firefox e no celular?

TESTE DE ACEITAÇÃO

- Executado pelo cliente ou usuário para decidir se aceita o Sistema (Quem decide é o usuário, não o programador).
- **Alfa:** no ambiente do desenvolvedor.
- **Beta:** no ambiente real do usuário.
- Baseado nos critérios de aceitação de cada história de usuário.
- Dado/Quando/Então: cada linha vira um passo de teste.

Dado que estou logado como bibliotecário

Quando registro o empréstimo de um livro disponível

Então o livro passa a constar como emprestado

TESTES DE CAIXA-PRETA E CAIXA-BRANCA

- Servem para encontrar tipos diferentes de problemas e são usados em momentos ou objetivos diferentes.
- **Caixa-preta:** “O sistema faz o que deveria fazer, do ponto de vista de quem usa?”
- **Caixa-branca:** “O código está internamente correto, completo e bem exercitado?”

Caixa-preta

Testa o software **sem conhecer ou considerar sua implementação interna** (observa entradas, saídas e regras de negócio.)

- Serve para validar funcionalidades
- Testar entradas validas e mensagens de erros
- Ajustar a detectar requisitos faltantes, ambiguidades ou falhas de integração

Caixa-branca

Testa o software **com conhecimento da estrutura interna do código** (funções, condições, laços, caminhos de execução e branches.)

- Serve para verificar a lógica interna e a estrutura do código
- Valida tratamento de exceções e fluxos internos.
- Detecta problemas de segurança

TÉCNICA PARTIÇÃO DE EQUIVALÊNCIA

Objetivo: reduzir a quantidade de testes sem perder a capacidade de encontrar defeitos.
Requisito: o campo idade aceita valores inteiros de 14 a 18.

Classe	Tipo	Valor representante
Menor que 14	Inválida	10
De 14 a 18	**Válida**	**16**
Maior que 18	Inválida	25
Não numérico	Inválida	abc

Valores da mesma classe devem ter o mesmo comportamento. Basta testar um representante de cada classe.

TÉCNICA ANÁLISE DE VALOR LIMITE

Objetivo: encontrar os defeitos nas bordas
Requisito: a senha deve ter de 8 a 20 caracteres.

Inválida	Válida	Inválida
0 a 7	8 a 20	21 ou mais



- Testar o mínimo, o máximo e um valor fora de cada borda.
- Detecta defeitos como > no lugar de >=, que a partição sozinha não encontra.

EXERCÍCIO: PARTIÇÃO E VALOR LIMITE

Requisito: a quantidade de um produto no estoque aceita números inteiros de 1 a 500.

1. Identifique as classes válidas e inválidas.
2. Escolha um representante de cada classe.
3. Liste os valores limite que devem ser testados.
4. Para cada valor, escreva o resultado esperado.

EXERCÍCIO: RESPOSTA

Técnica	Valores	Resultado esperado
Partição: menor que 1	-5	Recusa
Partição: de 1 a 500	250	Aceita
Partição: maior que 500	800	Recusa
Partição: não numérico	dez	Recusa
Valor limite inferior	0 e 1	Recusa 0, aceita 1
Valor limite superior	500 e 501	Aceita 500, recusa 501

Com 8 valores cobrimos as classes e as bordas do requisito.

CAIXA-BRANCA: COBERTURA

```
function desconto(float $total, bool $aluno): float {
  $d = 0;
  if ($aluno) $d = 0.10;
  if ($total > 100) $d += 0.05;
  return $total * (1 - $d);
}
```

Critério	Exige	Testes necessários
Cobertura de comandos	Cada linha executada uma vez	(150, true)
Cobertura de decisões	Cada if verdadeiro e falso	(150, true) e (50, false)

100% de comandos não garante 100% de decisões. O caso sem desconto nunca seria testado.

TESTE DE REGRESSÃO

- **Regressão:** falha que surge em uma funcionalidade que já funcionava, depois de uma alteração no código.
- **Teste de regressão:** executar de novo os casos de teste já aprovados após cada correção ou mudança.
- Por isso os casos de teste são escritos e guardados, e não apenas executados uma vez.



Testes automatizados tornam a regressão barata: rodam todos em segundos.

03

Teste na Prática

Planejar, executar, registrar e corrigir

ESTRUTURA DE UM CASO DE TESTE

1. **ID**
   CT01, CT02...
2. **Objetivo**
   O que está sendo testado
3. **Pré-condição**
   Estado necessário antes
4. **Entradas**
   Dados informados
5. **Passos**
   Ações, em ordem
6. **Resultado esperado**
   Definido pelos requisitos
7. **Resultado obtido**
   O que aconteceu
8. **Situação**
   Passou ou falhou

Itens 1 a 6 são escritos antes da execução. Itens 7 e 8, durante.

EXEMPLO DE CASO DE TESTE COMPLETO

ID	CT08
Objetivo	Verificar se o cadastro recusa senha abaixo do mínimo
Pré-condição	Usuário na tela de cadastro
Entradas	Nome: Ana | E-mail: ana@ifro.edu.br | Senha: 1234567
Passos	1. Preencher os campos 2. Clicar em Salvar
Resultado esperado	Mensagem "A senha deve ter de 8 a 20 caracteres" e nenhum registro gravado
Resultado obtido	Registro gravado sem mensagem
Situação	**Falhou**

CASOS DE TESTE: LOGIN (1/2)

ID	Objetivo	Entrada	Resultado esperado
CT01	Login válido	E-mail e senha cadastrados	Abre o menu inicial
CT02	Senha incorreta	Senha errada	Mensagem genérica de erro
CT03	E-mail inexistente	E-mail não cadastrado	Mesma mensagem do CT02

Pré-condição: existe o usuário teste@ifro.edu.br com senha Senha123.

CASOS DE TESTE: LOGIN (2/2)

ID	Objetivo	Entrada	Resultado esperado
CT04	Campos vazios	Nenhum dado	Pede o preenchimento
CT05	Acesso sem login	URL de página interna	Redireciona ao login
CT06	SQL Injection	' OR 1=1 --	Login recusado

Pré-condição: existe o usuário teste@ifro.edu.br com senha Senha123.

CASOS DE TESTE: CADASTRO (1/2)

ID	Objetivo	Entrada	Resultado esperado
CT07	Cadastro válido	Dados corretos e senha com 8 caracteres	Grava e lista o registro
CT08	Senha curta	Senha com 7 caracteres	Recusa e informa o mínimo
CT09	Senha longa	Senha com 21 caracteres	Recusa e informa o máximo

Pré-condição: o usuário está na tela de cadastro.

CASOS DE TESTE: CADASTRO (2/2)

ID	Objetivo	Entrada	Resultado esperado
CT10	E-mail inválido	joao.ifro.edu.br	Recusa e informa o formato
CT11	E-mail repetido	E-mail já cadastrado	Recusa sem duplicar
CT12	XSS	<script>alert(1)</script>	Exibe o texto sem executar

Pré-condição: o usuário está na tela de cadastro.

REGISTRO DE DEFEITO

Quando um caso de teste falha, o defeito é registrado para ser corrigido.

ID	BUG-03
Título	Cadastro aceita senha com 7 caracteres
Caso de teste	CT08
Passos para reproduzir	Abrir o cadastro, informar senha 1234567 e salvar
Esperado x obtido	Recusar a senha x registro gravado
Severidade	Alta
Evidência	Captura de tela da listagem com o registro

SEVERIDADE E PRIORIDADE

Severidade	Impacto	Exemplo
Crítica	Impede o uso ou expõe dados	Login sem senha por SQL Injection
Alta	Função principal errada	Cadastro grava senha inválida
Média	Função secundária errada	Busca ignora acentos
Baixa	Problema visual ou de texto	Botão desalinhado

- **Severidade:** o tamanho do estrago. Definida por quem testa.
- **Prioridade:** a ordem de correção. Definida pela equipe conforme o prazo.

CICLO DE VIDA DE UM DEFEITO



Registrado pelo testador

Desenvolvedor trabalha no código

Código alterado e enviado

Caso de teste executado de novo

Teste passou

- Se o reteste falhar, o defeito volta para **Aberto**.
- Ferramentas que controlam esse ciclo: GitHub Issues, Trello e Jira.
- Para um sistema de pequeno porte, uma planilha com ID, situação e responsável é suficiente.

TESTE AUTOMATIZADO COM PHPUNIT

```
$ composer require --dev phpunit/phpunit
```

```
<?php // tests/SenhaTest.php
use PHPUnit\Framework\TestCase;
class SenhaTest extends TestCase {
  public function testAceitaLimiteMinimo(): void {
    $this->assertTrue(validarSenha('12345678'));
  }
  public function testRecusaAbaixoDoMinimo(): void {
    $this->assertFalse(validarSenha('1234567'));
  }
}
```

```
$ vendor/bin/phpunit tests
OK (2 tests, 2 assertions)
```

QUANDO UM TESTE AUTOMATIZADO FALHA

```
$ vendor/bin/phpunit tests
F.
1) SenhaTest::testAceitaLimiteMinimo
Failed asserting that false is true.
tests/SenhaTest.php:5
FAILURES! Tests: 2, Assertions: 2, Failures: 1.
```

- `.` teste passou. **`F`** teste falhou.
- A saída informa qual teste falhou, o que era esperado e a linha do arquivo.
- Causa provável: a função usa `>` no lugar de `>=`. É o defeito que o valor limite revela.

ATIVIDADE PRÁTICA



Planejar

Escrevam no mínimo 10 casos de teste: login, cadastro e as duas funções principais do sistema, com pelo menos 2 de valor limite e 2 de segurança.



Executar

Executem os casos no sistema e preencham resultado obtido e situação, com captura de tela das falhas.



Corrigir e retestar

Registrem cada defeito, corrijam o código e executem de novo todos os casos (regressão).

Os resultados entram no bloco 6 do vídeo do projeto.

ENTREGA: RELATÓRIO DE TESTES

Item	Conteúdo	Pontos
Casos de teste	Mínimo de 10, com todos os campos preenchidos	15
Técnicas	Partição, valor limite e testes de segurança identificados	10
Registro de defeitos	ID, passos, esperado x obtido, severidade e evidência	10
Correção e reteste	Defeitos corrigidos e casos reexecutados	5
Total		40

O relatório vale 40% da nota da etapa.
O vídeo do projeto vale os outros 60%.

REFERÊNCIAS

DELAMARO, M. E.; MALDONADO, J. C.; JINO, M. Introdução ao teste de software. 2. ed. Rio de Janeiro: Elsevier, 2016.

MOLINARI, L. Testes de software: produzindo sistemas melhores e mais confiáveis. 4. ed. São Paulo: Érica, 2008.

PHPUNIT. PHPUnit Manual. Disponível em: https://docs.phpunit.de.

PRESSMAN, R. S. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, I. Engenharia de software. 9. ed. São Paulo: Pearson, 2011.